RISC-V: An Explanatory View into CPU Design

1st Marwan Negm
ITCS
Nile University
Cairo, Egypt
M.yasser2322@nu.edu.eg

2nd Ahmed Afifi
ITCS
Nile University
Cairo, Egypt
A.Mohamed2302@nu.edu.eg

3rd Steven Wilson
ITCS
Nile University
Cairo, Egypt
s.wilson2331@nu.edu.eg

4th Abdullah El-Dakhakhny
ITCS
Nile University
Cairo, Egypt
A.tarek2338@nu.edu.eg

5th Mark Hany
ITCS
Nile University
Cairo, Egypt
M.Hany2374@nu.edu.eg

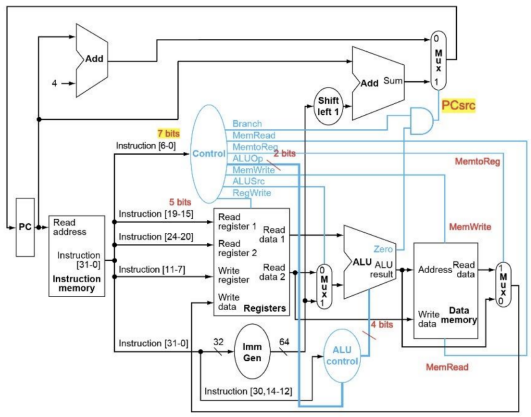

Fig. 1. RISC-V Single-Cycle Datapath

***Abstract*—This report documents the design and implementation of a 64-bit RISC-V Single Cycle CPU, exploring the nuances of designing a data-path and offering simulations and test-tables that rigorously confirm the correctness of the modules.**

***Index Terms*—RISC-V, CPU, Nile University.**

I. Introduction

Computers are a black box for the general user, but the architecture with which they are designed is a complex science worthy of exploration and scientific testing. The scope of this paper is limited to a RISC-V Single Cycle CPU (Central Processing Unit) due to resource and time constraints, it implements the modules necessary for a CPU to function: ALU, Control Unit, Register Files and Immediate Generator, Program Counter and Branching Logic, and finally a top down integration of the modules with the Instruction and Data memory.

II. Data Path Diagram

In order to implement the CPU, it is essential to analyze and dissect the data path flow that RISC-V utilizes for the CPU, as shown in Fig. 1.

III. Design Methodology

The design of the proposed RISC-V CPU follows a modular development methodology in which each hardware component is implemented and verified independently before being integrated into the complete system. A top-down integration strategy is then employed to ensure that the final processor conforms to the standard single-cycle RISC-V datapath. The datapath is decomposed into the following functional blocks: Arithmetic Logic Unit (ALU), Control Unit, Register File with Immediate Generator, Program Counter with Branch Logic, and the top-level integration incorporating the Instruction and Data Memories. Each module is described using Verilog HDL and validated through dedicated testbenches and simulation waveforms. The processor is instruction-centric in its design, as all internal modules are optimized to decode and execute instructions within a single clock cycle using a unified instruction format.

A. Design Choices

In this implementation, the Control Unit and the ALU Control Logic are merged into one module, as it prevents redundancy and simplify data path timing.

During the implementation of the Data Memory and the Instruction memory, it was determined that the addresses received from the ALU (through offset), must have a constraint set to be divisible by 8; as it is word addressed, and whether loading or storing, it would read/store a garbage value if it is split across two memory registers/double words.

IV. Control-Unit Truth Table

The complete control signal mapping for the implemented instructions is detailed in Table I. As shown in the table, the ALUCtrl signals differ based on the funct3 and funct7 fields for R-type instructions.

V. Modules

A. Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit is the computational core of the processor. It operates on two 64-bit operands and performs all

TABLE I
CONTROL UNIT TRUTH TABLE

Instr.	Opcode	funct3	funct7	Branch	MemR	MemW	MemToR	ALUSrc	RegWr	ALUCtrl
ADD	0110011	000	0	0	0	0	0	0	1	0010
SUB	0110011	000	1	0	0	0	0	0	1	0110
AND	0110011	111	X	0	0	0	0	0	1	0000
OR	0110011	110	X	0	0	0	0	0	1	0001
XOR	0110011	100	X	0	0	0	0	0	1	1100
SLL	0110011	001	X	0	0	0	0	0	1	1000
SRL	0110011	101	0	0	0	0	0	0	1	1001
SRA	0110011	101	1	0	0	0	0	0	1	1010
ADDI	0010011	000	X	0	0	0	0	1	1	0010
ANDI	0010011	111	X	0	0	0	0	1	1	0000
ORI	0010011	110	X	0	0	0	0	1	1	0001
XORI	0010011	100	X	0	0	0	0	1	1	1100
SLLI	0010011	001	X	0	0	0	0	1	1	1000
SRLI	0010011	101	0	0	0	0	0	1	1	1001
SRAI	0010011	101	1	0	0	0	0	1	1	1010
LD	0000011	X	X	0	1	0	1	1	1	0010
SD	0100011	X	X	0	0	1	0	1	0	0010
BEQ	1100011	000	X	1	0	0	0	0	0	0110

arithmetic and logical operations required by the supported RISC-V instruction set. The operation to be executed is selected using a 4-bit control signal generated by the Control Unit. The implemented functions include addition, subtraction, bitwise AND, OR, XOR, logical and arithmetic shifts.

Shift operations use the lower six bits of the second operand to determine the shift amount, allowing variable-length shifts suitable for 64-bit data. The ALU also generates a `Zero` flag which is asserted whenever the result of the operation is equal to zero. This signal is later used by the branch logic to decide whether a conditional branch should be taken. A default output value of zero is provided for undefined control codes to avoid unpredictable behavior.

1) Testing: The ALU was validated using a dedicated simulation that applied a variety of input patterns and control signals. Arithmetic operations were checked by performing addition and subtraction on known operands, including a subtraction case where both inputs were equal to verify correct assertion of the zero flag. Logical operations were tested using distinct hexadecimal patterns to confirm correct behavior for AND, OR, and XOR. Shift instructions were also evaluated by shifting a single set bit by different amounts using the lower bits of the second operand. The observed results matched the expected values in all cases, and the zero flag behaved correctly whenever the ALU output was zero. These tests confirm that the ALU performs all required operations reliably.

B. Control Unit & ALU Control Logic

The Control Unit is responsible for generating all internal control signals required for instruction execution. In this design, the Control Unit and the ALU Control Logic are merged into a single module for simplicity and efficiency. This module decodes the opcode, funct3, and funct7 fields from the instruction to produce the following control signals: `Branch`, `MemRead`, `MemWrite`, `MemToReg`, `ALUSrc`, `RegWrite`, and `ALUCtrl`.

R-type and I-type instructions are handled by selecting the appropriate ALU operation based on `funct3` and `funct7` fields, while memory and branch instructions are controlled primarily by the opcode. For instance, load instructions (`LD`) enable `MemRead` and `MemToReg`, store instructions (`SD`) assert `MemWrite`, and branch instructions (`BEQ`) assert the `Branch` signal and configure the ALU for subtraction to compare register values.

Table I summarizes the complete control signal mapping for the supported instructions. Each row shows the combination of opcode, funct3, and funct7 fields along with the resulting control signals, illustrating how the CPU correctly interprets and executes each instruction in a single clock cycle.

1) Testing: The Control Unit was tested with all supported instructions, including R-type (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA), I-type (ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI), load/store (LD, SD), and branch (BEQ). For each instruction, the outputs Branch, MemRead, MemToReg, MemWrite, ALUSrc, RegWrite, and ALUCtrl were checked against expected values. All tests confirmed correct behavior. R-type and I-type instructions set the ALU and register signals correctly, load/store instructions handled memory operations as intended, and the branch instruction correctly activated branch control. Overall, the testing verified that the Control Unit reliably produces the correct control signals for the implemented single-cycle RISC-V CPU.

C. Register File & Immediate Generator

The Register File stores all general-purpose registers required by the RISC-V CPU. It consists of 32 registers, each 64 bits wide. Two registers can be read simultaneously, while one register can be written per clock cycle. The register

at index zero is hardwired to zero, ensuring that any reads from this register always return zero, and writes to it are ignored. The write operation is synchronous with the rising edge of the clock and is only performed when the `RegWrite` signal is asserted. This arrangement allows instructions to read operands and write results efficiently while respecting the RISC-V ISA conventions.

The Immediate Generator is responsible for extracting and sign-extending immediate values from the instruction. Different instruction types use different bit fields for the immediate. I-type and load instructions use bits 31 to 20, S-type instructions use bits 31 to 25 combined with 11 to 7, and B-type (branch) instructions combine bits 31, 7, 30 to 25, and 11 to 8 with a zero appended to form the least significant bit. The Immediate Generator takes the instruction as input and produces a 64-bit sign-extended immediate suitable for arithmetic, memory, or branch calculations. This design ensures that all immediate values are correctly formatted for use in the ALU or branch logic.

1) Testing: A testbench was developed to verify the ImmGen module by applying representative RISC-V instructions of types addi, ld, sd, and beq. Each instruction was fed into the module, and the extracted immediate value was compared with the expected result.

All test cases passed successfully, confirming that the ImmGen correctly decodes immediate values for arithmetic, load, store, and branch instructions.

D. Program Counter, Branch Logic & PC Update

The Program Counter (PC) keeps track of the address of the instruction currently being executed. It is implemented as a 64-bit register that updates on the rising edge of the clock. A synchronous reset is included to set the PC back to zero at the start or when required. The input to the PC is the next instruction address, which comes from a multiplexer selecting between the sequential address (PC + 4) and the branch target address. The output of the PC goes directly to the instruction memory and to other modules that require the current program address. The design ensures that instruction flow can continue sequentially or jump to a branch target as needed.

1) Testing: The Program Counter was tested using a simulation that mimicked the clock and instruction flow. Initially, the PC was reset to verify it starts at zero. Then, normal sequential execution was observed, with the PC incrementing by four each cycle. Branch instructions were tested next. When the branch signal was active but the ALU zero flag was low, the PC correctly continued to the next sequential address, showing that branches are not taken when the condition is false. When the zero flag was high, the PC jumped to the branch target address as expected. After branching, the PC resumed normal sequential updates. Monitoring these scenarios confirmed that the PC correctly handles reset, sequential execution, and branching in all tested cases.

E. Instruction Memory, Data Memory & Top-Level Integration

The RISKV_CPU serves as the top-level module that orchestrates the entire processor by interconnecting the Control Unit, ALU, Register File, and memory units to execute the single-cycle datapath. It drives the Instruction Memory, which is responsible for taking the program counter address and outputting the corresponding 32-bit instruction from a preloaded file. Finally, the processor interacts with the Data Memory to handle load and store operations, ensuring data is accurately read or written synchronously based on the calculated address and control signals.

1) Testing: Testing was done on a test program provided by Nile University, the wave form is provided for this test program in Simulation Waveforms.

TABLE II
EXPECTED OUTCOMES FOR REGISTERS AND MEMORY AFTER EXECUTION

Register / Memory	Expected Value (Decimal)
x0	0
x17	1632
x5	0
x15	1632
x4	1
mem[0]	1632

VI. SIMULATION WAVEFORMS

The waveforms for all the components are for custom test benches designed by our team, but the actual outcomes from the test program provided by Nile University match the expected outcomes.

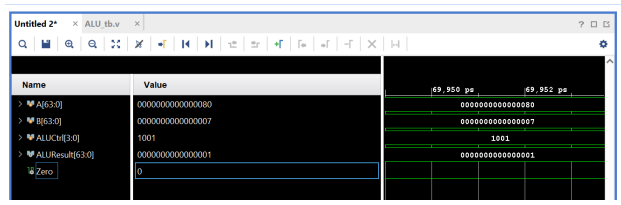

Fig. 2. ALU operation waveform.

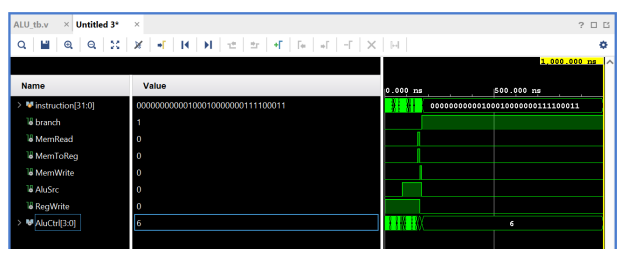

Fig. 3. Control Unit signals waveform.

VII. CONCLUSION

This report presented the design and implementation of a 64-bit single-cycle RISC-V CPU. Each component, including the ALU, Control Unit, Register File, Immediate Generator, Program Counter, and memory modules, was developed, tested, and verified independently before integration. The CPU correctly executes arithmetic, logical, load/store, and branch instructions in a single clock cycle.

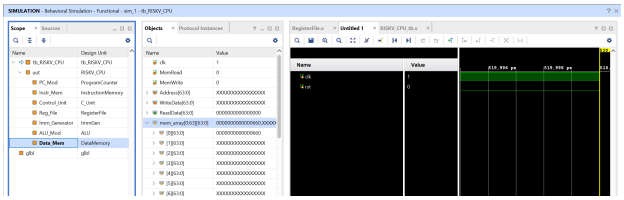

Fig. 4. Simulation waveform part 1.

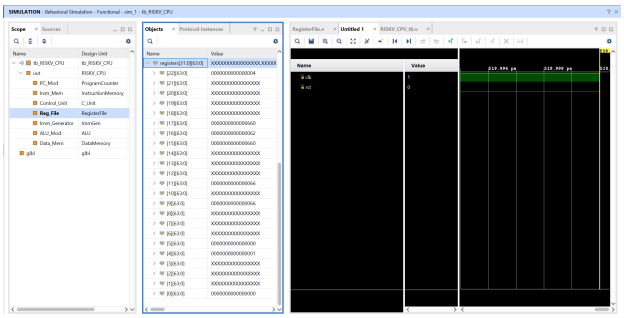

Fig. 5. Simulation waveform part 2.

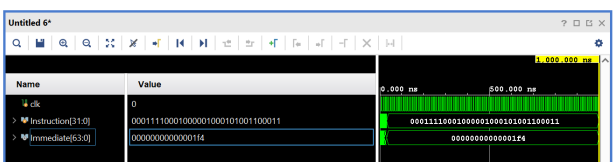

Fig. 6. Immediate Generator waveform.

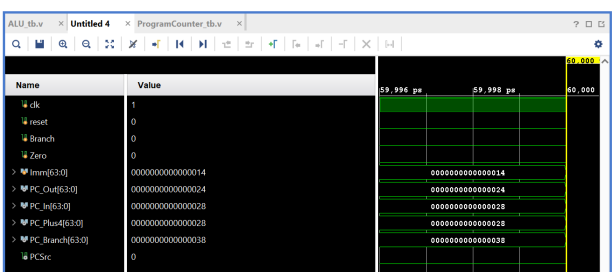

Fig. 7. Program Counter waveform.

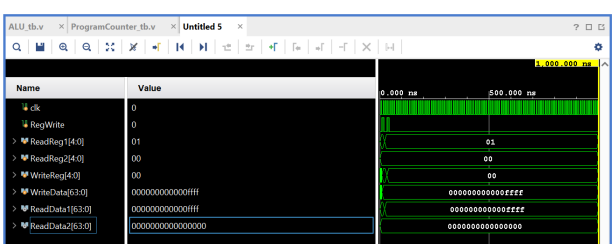

Fig. 8. Register File waveform.

Simulation and testbench results confirmed the correctness of all modules and the complete system, including immediate value extraction, ALU operations, control signal generation, and memory access. The integrated processor executed the test program as expected, with registers and memory holding the correct values. This work demonstrates a functional single-cycle RISC-V CPU implemented in Verilog and provides a foundation for future improvements such as pipelining or additional instruction support.

VIII. References